Molecular dynamics: 2D Lennard-Jones Simulation

Plabon Shaha and Guillem Masdemont
University of Ljubljana, Spring Semester, EMAI Cohort 25–27

May 3, 2026

Implementation We parallelize the algorithm and present several versions that are built on top of each other. We compare the speedup performance with the baseline model. We consider the following 4 improved models:

- **V1 – CPU baseline:** All computation runs sequentially on the CPU. Forces are computed with an $O(n^2)$ nested loop, counting each pair twice.
- **V2 – GPU forces, 2D grid:** Force computation is offloaded to the GPU using a 2D thread grid where each thread handles one (i, j) pair. Integration stays on the CPU, and GPU memory is allocated and freed every step.
- **V3 – GPU forces, 1D pairs:** Force computation uses a 1D kernel with one thread per unique pair and we apply Newton's third law to halve the work. The integration still runs on the CPU with per-step memory transfers.
- **V4 – Fully GPU, AoS:** All steps — forces, integration, and energy computation — run on the GPU. Memory is allocated once and data never leaves the device during the simulation loop.
- **V5 – Fully GPU, SoA:** Identical to V4 but uses a Structure-of-Arrays memory layout, enabling coalesced memory access across warps.

The results show consistent speedups as optimisations are added across all particle counts. V3 is roughly 2× faster than V2, primarily due to halving the number of force evaluations. The largest gain comes from eliminating CPU-GPU transfers in V4, which is 3–4× faster than V3 at small N and around 2× at $N = 8000$. V5 adds a further 1.03–1.32× improvement over V4, with the gain growing with N as coalesced memory access becomes more beneficial at larger problem sizes. Overall, V5 achieves a 3.6–8.1× speedup over V2 depending on particle count. To make results meaningful, we also study the standard deviation. We observed a high standard deviation in V2, which is likely due to variable overhead from repeated memory allocation and deallocation each step.